STREETLY

Platform Documentation

Street-Level Business Discovery · Field Agent Network
Delivery Logistics · Local Marketplace

Version 1.0 · July 2026

TABLE OF CONTENTS

1. PLATFORM OVERVIEW

Streetly is a hyper-local business discovery and field marketing platform built for the Nigerian market. It organises businesses at street level (City !' Area !' Street), empowers a distributed network of field agents to crowdsource verified data, and layers a fully-integrated delivery and marketplace system on top. The platform connects four primary stakeholders: customers (visitors), business owners, field agents, and delivery riders — all coordinated through a tiered staff hierarchy of scout managers, regional managers, moderators, and administrators.

Core Value Proposition

- Street-level discovery: find any business by city, area, and exact street.
- Verified listings crowdsourced by an incentivised network of field agents.
- Embedded marketplace: businesses can sell products and dispatch riders in one flow.
- Agent earnings system: leaderboard, commission tracking, and withdrawal management.
- End-to-end delivery tracking with real-time rider location updates.
- Multi-tier staff hierarchy for scalable operations management.

Key Metrics Tracked

- Total businesses, pending approvals, rejected listings.
- Active agents, total earnings, available balances.
- Delivery orders — placed, in-transit, delivered, cancelled.
- Rider availability, vehicle types, and proximity.
- Support ticket volume and resolution rate.
- Platform analytics: page views, search queries, category trends.

2. ARCHITECTURE & TECHNOLOGY STACK

Monorepo Structure

The project is a pnpm workspace monorepo containing the following packages:

Package	Type	Description
artifacts/streetly	Frontend (React)	Main SPA — all customer, agent, owner, and admin UI
artifacts/api-server	Backend (Node.js)	REST API server — auth, business logic, database access
lib/db	Library	Drizzle ORM schema, migrations, and database client
lib/api-spec	Library	OpenAPI 3.0 specification (single source of truth)
lib/api-zod	Library	Zod schemas auto-generated from the OpenAPI spec

Technology Stack

- React 18 + TypeScript (frontend)
- Vite (build tool & dev server)
- Wouter (client-side routing)
- TanStack Query (server state)
- Tailwind CSS + shadcn/ui
- Framer Motion (animations)
- Lucide React (icons)
- Node.js + Express (API server)
- PostgreSQL (primary database)
- Drizzle ORM (type-safe queries)
- Pino (structured logging)
- JWT (stateless auth tokens)
- Nodemailer (transactional email)
- esbuild (API server bundler)

Request Flow

Browser !' Vite dev proxy (development) / Replit reverse proxy (production) !' Express API server !' Drizzle ORM !' PostgreSQL. All authenticated requests carry a Bearer JWT in the Authorization header. The frontend reads BASE_URL from import.meta.env to construct API URLs correctly in both environments.

Database
PostgreSQL is provided as a Replit-managed database. The connection string is available via the DATABASE_URL environment variable. Schema migrations are managed with Drizzle Kit.

3. USER ROLES & PERMISSIONS

Streetly implements a role-based access control model. Each user has exactly one role stored in the users table. Staff roles (moderator, scout_manager, regional_manager, admin) use dedicated portal pages with separate authentication flows.

Role Overview

Role	Portal	Description
visitor	/ (public)	Default role for all registered accounts. Can browse, favourite, message businesses.
business_owner	/owner-dashboard	Manages one or more business listings, marketplace items, and delivery orders.
field_agent	/agent-dashboard	Submits business listings, tracks earnings, requests withdrawals.
delivery_rider	/rider-dashboard	Accepts and fulfils delivery orders. Tracks earnings.
moderator	/moderator	Reviews and approves business listings, handles support tickets.
scout_manager	/scout-manager	Manages field agents, approves agent applications, oversees commissions.
regional_manager	/regional-manager	Oversees all agents assigned to a region; views earnings and business data.
admin	/admin	Full platform control — users, agents, riders, businesses, settings, analytics.

Visitor (Public User)

- Browse businesses by city, area, and street without authentication.
- Register for an account (role assigned during registration).
- View business profiles, photos, and marketplace storefronts.
- Submit contact forms and leave reviews.
- Track delivery orders via a shareable tracking URL.

Business Owner

- Onboard a new business or claim an existing auto-generated listing.
- Edit business profile: name, description, address, category, contact, hours, photos.
- Manage marketplace items: add products with photos, prices, and stock.
- Receive and manage delivery orders placed through the storefront.
- View earnings and commission history.
- Chat with customers through the built-in messaging system.

Field Agent

- Submit an application with full name, ID type/number, bank details, and passport photo.
- After admin approval, submit new business listings for review.
- Earn commission for each approved listing; track on a personal dashboard.
- Climb the agent leaderboard based on approved listing count.
- Request withdrawal of available balance to a registered bank account.
- View a NIN slip document for identity verification.

Delivery Rider

- Apply with vehicle type (bicycle, motorcycle, car, van) and bank details.
- Toggle online/offline status. Riders only receive orders when online.
- Receive real-time delivery requests from nearby businesses.
- Update order status: accepted !’ picked_up !’ delivered.

Earn per delivery; track cumulative earnings and withdrawal status.

4. AUTHENTICATION & SECURITY

Token Format

Streetly uses a lightweight JWT-style token: a base64-encoded JSON payload containing userId and exp (expiry timestamp). Tokens are valid for 7 days from issuance. They are stored in localStorage under the key streetly_token.

Auth Endpoints

Method	Path	Description
POST	/api/auth/register	Create account. Returns token + role on success.
POST	/api/auth/login	Authenticate with email + password. Returns token + role.
GET	/api/auth/me	Returns current user profile (id, name, email, role).

Route Protection Pattern

Each protected frontend page manually reads the token from localStorage and calls GET /api/auth/me on mount. Based on the returned role, the page either allows access or redirects to the correct portal. There is no global middleware on the frontend — each page enforces its own access rules.

- No token !' redirect to login page or show AdminLoginGate.
- Wrong role (e.g. field_agent on /admin) !' redirect to the correct portal for that role.
- Valid token, correct role !' page renders normally.

Backend Middleware

The API server uses a requireAnyRole(...roles) middleware factory:

- All /admin/* routes require at least admin, moderator, or scout_manager.
- Admin-only sub-routes (/admin/users, /admin/stats, /admin/kyc, etc.) restrict further to admin only.
- Scout manager sub-routes (/admin/agents, /admin/properties, /admin/withdrawals) allow scout_manager.
- Moderator sub-routes (/admin/businesses, /admin/support-tickets) allow moderator.
- /regional-manager/* routes require regional_manager role.

Impersonation (Admin Only)

Admins can generate a short-lived access token for any user via POST /api/admin/impersonate/:userId. The returned token allows the admin to log in as that user and experience their portal directly. A green banner is shown while impersonating, with a one-click button to navigate to the impersonated user's portal.

5. STREET EXPLORER & DISCOVERY

Hierarchy

Streetly organises the physical world in three tiers: City !’ Area !’ Street. Every business is associated with a street, which belongs to an area, which belongs to a city. The explorer lets visitors drill down through this hierarchy to find businesses at a hyper-local level.

Discovery Endpoints

Method	Path	Description
GET	/api/categories	All business categories (id, name, slug, icon).
GET	/api/cities	All cities on the platform.
GET	/api/cities/:id/areas	All areas within a city.
GET	/api/areas/:id/streets	All streets within an area.
GET	/api/streets/:id/businesses	All approved businesses on a street.
GET	/api/businesses/featured	Businesses in the featured/promoted order.
GET	/api/businesses/search	Full-text search by name, category, or street.
GET	/api/geo/info	IP-based geolocation: returns detected city and currency.

Search & Filtering

- Search by business name, category name, or street name.
- Filter by category slug (e.g. ?category=restaurants).
- Filter by city or area ID.
- Results include business photo, category, street, area, and contact info.
- Featured businesses appear at the top of listing results.

6. BUSINESS LISTINGS

Business Lifecycle

A business goes through the following states: pending !' approved / rejected. Agents submit new businesses which are held in pending status until reviewed by a moderator or admin. Business owners can also onboard directly; their listing also requires approval. Rejected businesses can be resubmitted with corrections.

Status	Visibility	Who Can Change
pending	Owner/Agent only	Moderator or Admin approve/reject
approved	Public	Admin can suspend
rejected	Owner/Agent only	Can be resubmitted
suspended	Hidden from public	Admin only can reinstate

Business Data Fields

- Name
- Slug (URL-friendly, auto-generated)
- Category
- Street / Area / City
- Description
- Phone number
- WhatsApp number
- Opening hours
- Photos (multiple)
- Latitude / Longitude
- Plan (free / premium)
- Agent ID (who submitted)
- Owner user ID (after claim)
- Status (pending/approved/rejected/suspended)

Business Claim Flow

When an agent submits a business, the listing is created without an owner. The actual business owner can later claim it by verifying ownership. The claim goes through admin approval. Once approved, the business is linked to the owner's account and they gain full edit access.

Business API Endpoints

Method	Path	Auth	Description
GET	/api/businesses	Public	List all approved businesses (with filters).
POST	/api/businesses	Agent/Owner	Submit a new business listing.
GET	/api/businesses/:id	Public	Get single business profile.
PATCH	/api/businesses/:id	Owner/Admin	Update business details.
GET	/api/businesses/mine	Owner	List all businesses owned by current user.
POST	/api/businesses/:id/photos	Owner	Upload business photo.
DELETE	/api/businesses/:id/photos/:photoId	Owner	Remove a business photo.
POST	/api/businesses/:id/claim	Visitor	Submit an ownership claim.

7. FIELD AGENT SYSTEM

Agent Onboarding

To become a field agent, a user registers with the field_agent role and completes an application form. The application captures personal details, bank account information for payouts, and two identity documents: a passport photograph and a NIN (National Identification Number) slip. Applications are reviewed and approved by a scout manager or admin.

Application Fields

Field	Required	Notes
Full Name	Yes	Legal name as on ID
ID Type	Yes	e.g. NIN, Driver's License, Voter's Card
ID Number	Yes	The ID document number
Passport Photo	Yes	Stored as base64 data URL
NIN Slip Image	Yes	Stored as base64 data URL
Bank Name	Yes	Nigerian bank name
Account Number	Yes	10-digit NUBAN number
Account Name	Yes	Name on bank account
Phone Number	Yes	Contact number
Address	No	Home/work address

Agent Dashboard

- Overview: total earnings, available balance, total listings submitted.
- Listing history: each submitted business with approval status.
- Withdrawal requests: request payout, view pending/completed withdrawals.
- Leaderboard: rank among all agents by approved listing count.
- Document view: view passport photo and NIN slip on file.

Commission & Earnings

Agents earn a commission for each approved business listing. Earnings accumulate in the agent's available_balance. Withdrawal requests are reviewed by a scout manager or admin. Approved withdrawals are transferred to the agent's registered bank account.

Agent API Endpoints

Method	Path	Description
POST	/api/agents/apply	Submit agent application.
GET	/api/agents/:id/dashboard	Agent dashboard data (earnings, listings, withdrawals).
GET	/api/agents/leaderboard	Top agents ranked by approved listings.
POST	/api/agents/:id/withdraw	Request a withdrawal of available balance.
GET	/api/admin/agents/all	Admin/Scout: list all agents with status.
PATCH	/api/admin/agents/:id/approve	Admin/Scout: approve or reject an agent.

Method	Path	Description
PATCH	/api/admin/agents/:id/suspend	Admin/Scout: suspend or reinstate an agent.

Regional Manager Assignment

Admins can assign agents to a regional manager. Once assigned, the regional manager can view all agents in their portfolio, inspect each agent's business listings, earnings, and commissions, and monitor overall performance within their region.

8. DELIVERY RIDER SYSTEM

Rider Onboarding

Riders register with the delivery_rider role and complete an application with vehicle type, bank details, and personal information. Applications require admin approval. Once approved, riders can go online to start receiving delivery requests.

Vehicle Types

- Bicycle
- Motorcycle (most common)
- Car
- Van

Delivery Order Lifecycle

Status	Triggered By	Description
pending	Business/Customer	Order placed, awaiting rider acceptance.
accepted	Rider	Rider has accepted the order.
picked_up	Rider	Rider has collected the items from the business.
delivered	Rider	Items delivered to the customer.
cancelled	Business or System	Order cancelled before pickup.

Rider Features

- Toggle online/offline status from rider dashboard.
- Update GPS location while online (used for proximity calculation).
- View incoming delivery requests with fee and distance info.
- Accept orders and update status through the delivery flow.
- Earnings tracked per delivery; withdrawal to bank account.

Rider API Endpoints

Method	Path	Description
POST	/api/riders/apply	Submit rider application.
PATCH	/api/riders/status	Toggle online/offline status.
PATCH	/api/riders/location	Update GPS coordinates.
GET	/api/riders/nearby	Get nearby available riders (for business dispatch).
GET	/api/deliveries/:id	Get delivery order details and tracking info.
PATCH	/api/deliveries/:id/status	Update delivery status (rider).

9. LOCAL MARKETPLACE

Overview

Every approved business can activate a marketplace storefront. Business owners add products with photos, descriptions, prices, and stock levels. Customers can browse the storefront and place orders. Orders trigger a delivery request that is dispatched to a nearby rider.

Marketplace Item Fields

- Name
- Description
- Price (NGN)
- Stock quantity
- Photo (base64)
- Category/tags
- Availability status
- Business ID (parent)

Marketplace Flow

- 1. Business owner creates marketplace items in the owner dashboard.
- 2. Customer visits /[slug]/store and adds items to an order.
- 3. Customer provides delivery address and confirms order.
- 4. System calculates delivery fee based on distance.
- 5. Available nearby rider is found and notified.
- 6. Rider accepts !’ picks up !’ delivers.
- 7. Order status updates in real time; customer can track via /deliveries/:id.

Marketplace API Endpoints

Method	Path	Description
GET	/api/businesses/:id/marketplace-items	List marketplace items for a business.
POST	/api/businesses/:id/marketplace-items	Add a new marketplace item (owner).
PATCH	/api/marketplace-items/:id	Update item details or availability.
DELETE	/api/marketplace-items/:id	Remove marketplace item.
GET	/api/businesses/:id/available-riders	Find nearby available riders.
POST	/api/deliveries	Create a delivery order.
GET	/api/businesses/:id/marketplace-orders	List all orders for a business.

10. PROPERTY LISTINGS

Overview

Streetly includes a dedicated module for vacant property listings (residential and commercial). Any registered user can submit a property. Listings require admin/moderator approval before appearing publicly. The property module supports rental, lease, and sale listings.

Property Fields

- Title
- Description
- Address
- City / Area / Street
- Property type (residential/commercial)
- Price amount + price type (rent/sale/lease)
- Bedrooms / Bathrooms
- Contact name + phone
- Photos (multiple)
- Status (pending/approved/rejected)
- Agent ID (submitter)
- Created date

Property API Endpoints

Method	Path	Description
GET	/api/properties	List all approved property listings.
POST	/api/properties	Submit a new property listing.
GET	/api/properties/:id	Get property details with photos.
POST	/api/admin/properties/:id/approve	Admin/Scout: approve or reject property.

11. ADMIN DASHBOARD

Access

The admin dashboard is accessible at /admin. It uses a dedicated login gate that accepts only users with the admin role. Admin credentials are seeded on server startup via ensureAdminUser().

Dashboard Sections

Section	Description
Analytics	Platform-wide KPIs: total businesses, agents, riders, users, revenue.
Businesses	Review, approve, reject, suspend, and edit all business listings.
KYC	View agent passport photos and NIN slips for identity verification.
Agents	List all agents; approve, reject, suspend; view earnings and listings.
Properties	Review and approve vacant property submissions.
Riders	List all riders; approve, reject, suspend.
Claims	Review business ownership claim requests.
Deliveries	Monitor all delivery orders across the platform.
Withdrawals	Approve or reject agent withdrawal requests.
Staff Accounts	Manage moderators, scout managers, and regional managers.
All Users	Full user registry with role, status, and account actions.
Featured Order	Drag-and-drop reordering of featured/promoted businesses.
Messages	Admin broadcast messaging and system notifications.
Gallery	PIN-locked image archive (PIN: 1537) of all uploaded photos.
Export	Export platform data as CSV/JSON.
Settings	Platform configuration: SMTP, fees, and general settings.

Staff Account Management

Admins can create and manage accounts for moderators, scout managers, and regional managers. Each staff member has a dedicated portal (/moderator, /scout-manager, /regional-manager). The admin can also use "Login As" to impersonate any staff member or regular user for testing and support.

Gallery (PIN-locked)

The gallery section aggregates all uploaded images across the platform into one view, organised into three folders:

- Passports — all agent passport photos with agent name.
- NIN Slips — all agent NIN document images.
- Business — all photos uploaded to business listings.

Click any image to open a full-screen lightbox. The gallery is protected by PIN: 1537.

12. STAFF PORTALS

Moderator Portal (/moderator)

Moderators focus on content quality and customer support.

- Review and approve/reject pending business listings.
- Manage featured business order.
- Handle support tickets: view, reply, resolve.
- View and moderate business categories.
- Cannot access user accounts, financial data, or system settings.

Scout Manager Portal (/scout-manager)

Scout managers oversee the field agent network.

- View all agent applications (pending, approved, suspended).
- Approve, reject, or suspend field agents.
- Monitor agent commissions and earnings.
- Review and approve withdrawal requests.
- Manage business categories.
- Review and approve property listings.

Regional Manager Portal (/regional-manager)

Regional managers have a portfolio of assigned field agents.

- View all agents assigned to them by an admin.
- Drill into each agent's profile: businesses submitted, status, earnings.
- View all business listings submitted by agents in their portfolio.
- Monitor commission and withdrawal history per agent.
- Read-only view — cannot approve/reject applications.

Cross-Portal Redirect Guards

Each portal page validates the token on mount. If the token belongs to a different role, the user is immediately redirected to the correct portal for their role. This prevents accidental access to wrong portals from saved bookmarks or direct URL entry.

From Portal	If Role Is	Redirect To
/admin	moderator	/moderator
/admin	scout_manager	/scout-manager
/admin	regional_manager	/regional-manager
/admin	delivery_rider	/rider-dashboard
/scout-manager	admin	/admin
/scout-manager	moderator	/moderator
/moderator	scout_manager	/scout-manager
/regional-manager	scout_manager	/scout-manager

13. SUPPORT & MESSAGING

Support Ticket System

Any registered user can open a support ticket from the /support page. Tickets have a subject, category, and initial message. Moderators and admins can view, reply to, and resolve tickets from their respective dashboards.

Ticket Status	Description
open	Newly created; awaiting staff response.
in_progress	Staff has replied and is actively working on it.
resolved	Issue closed by staff.

Messaging (Chat)

Streetly includes a real-time conversation system between customers, business owners, and riders. Conversations are associated with a business and optionally a delivery order. Messages carry a sender_role field (customer, business, rider) for display context.

- Customers can initiate a conversation from a business profile page.
- Business owners view all conversations in the owner dashboard.
- Admin can view all system messages from the Messages section.

Email Notifications (SMTP)

Outbound emails (account verification, approval notifications, withdrawal confirmations) are sent via Nodemailer. SMTP settings (host, port, user, password, from address) are configured in the admin Settings section and stored in the settings table. The mailer reads these settings on each send; if unconfigured it logs a warning and skips sending rather than crashing.

14. API REFERENCE

Base URL

Development: http://localhost:8080 | Production: https://mystreetly.app (reverse-proxied)

All API routes are prefixed with /api.

Authentication Endpoints

Method	Path	Body	Response
POST	/api/auth/register	{ name, email, password, role }	{ token, user }
POST	/api/auth/login	{ email, password }	{ token, user }
GET	/api/auth/me	(Bearer header)	{ id, name, email, role }

Discovery Endpoints

Method	Path	Auth	Response
GET	/api/categories	None	Category[]
GET	/api/cities	None	City[]
GET	/api/cities/:id/areas	None	Area[]
GET	/api/areas/:id/streets	None	Street[]
GET	/api/streets/:id/businesses	None	Business[]
GET	/api/businesses/featured	None	Business[] (ordered)
GET	/api/businesses/search?q=…	None	Business[]
GET	/api/geo/info	None	{ city, currency, country }

Business Endpoints

Method	Path	Auth	Notes
GET	/api/businesses	None	Supports ?category, ?city, ?area filters
POST	/api/businesses	Agent/Owner	Creates pending listing
GET	/api/businesses/:id	None	Full profile with photos
PATCH	/api/businesses/:id	Owner/Admin	Update name, desc, photos, etc.
DELETE	/api/businesses/:id	Admin	Permanently removes listing
GET	/api/businesses/mine	Owner	All businesses for current user
POST	/api/businesses/:id/claim	Any Auth	Submit ownership claim

Agent Endpoints

Method	Path	Auth	Notes
POST	/api/agents/apply	field_agent	Submit application with ID docs
GET	/api/agents/:id/dashboard	field_agent	Earnings, listings, withdrawals

Method	Path	Auth	Notes
POST	/api/agents/:id/withdraw	field_agent	Request payout
GET	/api/agents/leaderboard	None	Top 20 agents by listings
GET	/api/agents/:id/listings	None	Businesses submitted by agent

Admin Endpoints

Method	Path	Allowed Roles	Description
GET	/api/admin/stats	admin	Platform-wide statistics
GET	/api/admin/agents/all	admin, scout_manager	All agents
PATCH	/api/admin/agents/:id/approve	admin, scout_manager	Approve/reject agent
GET	/api/admin/businesses/all	admin, moderator	All businesses
PATCH	/api/admin/businesses/:id/approve	admin, moderator	Approve/reject business
GET	/api/admin/riders/all	admin	All riders
GET	/api/admin/users/all	admin	All platform users
GET	/api/admin/kyc	admin	Agent passport + NIN data
GET	/api/admin/withdrawals	admin, scout_manager	Pending withdrawals
POST	/api/admin/impersonate/:id	admin	Generate impersonation token
GET	/api/admin/gallery	admin, moderator, scout_manager	All uploaded images
GET	/api/admin/analytics	admin	Usage analytics
GET	/api/admin/settings	admin	Platform config
PATCH	/api/admin/settings	admin	Update platform config

15. DATABASE SCHEMA

users

Column	Type	Notes
id	serial PK	Auto-increment primary key
name	text	Full display name
email	text UNIQUE	Login email
password	text	bcrypt hash
role	user_role enum	visitor | business_owner | field_agent | delivery_rider | moderator | scout_manager | regional_manager | admin
msaId	text UNIQUE	Internal MSA identifier
isSuspended	boolean	Account suspension flag
creditPoints	integer	Loyalty/referral credits
referralCode	text UNIQUE	Shareable referral code
createdAt	timestamp	Account creation time

businesses

Column	Type	Notes
id	serial PK	
name	text	Business display name
slug	text UNIQUE	URL-friendly identifier (auto-generated)
description	text	About the business
categoryId	integer FK!'categories	
streetId	integer FK!'streets	
agentId	integer FK!'agents	Who submitted this listing
ownerId	integer FK!'users	Set after ownership claim is approved
managerId	integer FK!'users	Assigned regional manager
status	text	pending | approved | rejected | suspended
plan	text	free | premium
phone	text	
whatsapp	text	
lat / lng	numeric	GPS coordinates
createdAt	timestamp	

agents

Column	Type	Notes
id	serial PK	
userId	integer FK!'users	One-to-one with user account

Column	Type	Notes
fullName	text	Legal name
idType / idNumber	text	Government ID details
passportPhoto	text	base64 data URL
ninSlip	text	base64 NIN document image
bankName / accountNumber / accountName	text	Payout bank details
status	text	pending | approved | rejected | suspended
totalEarnings / availableBalance	numeric	Financial tracking
createdAt	timestamp	

Other Key Tables

Table	Key Columns	Purpose
categories	id, name, slug, icon	Business classification
cities	id, name, state	Top-level geography
areas	id, name, cityId	Neighbourhood within a city
streets	id, name, areaId	Street within an area
business_photos	id, businessId, url	Multiple photos per business
riders	id, userId, vehicleType, status, lat, lng	Rider profiles and location
delivery_orders	id, businessId, riderId, status, fee, address	Orders with full lifecycle
delivery_order_items	id, orderId, itemId, quantity, price	Line items per order
marketplace_items	id, businessId, name, price, stock, photo	Products for sale
vacant_properties	id, title, address, priceAmount, priceType, status	Real estate listings
business_claims	id, businessId, userId, status	Ownership claim workflow
withdrawals	id, agentId, amount, status	Agent payout requests
support_tickets	id, userId, subject, status	Customer service tickets
support_ticket_replies	id, ticketId, userId, message	Reply thread
conversations	id, businessId, customerId	Chat session
chat_messages	id, conversationId, senderRole, content	Individual messages
settings	key, value	Key-value platform config (SMTP, fees, etc.)
analytics	id, event, metadata, createdAt	Platform usage events

16. DEPLOYMENT & ENVIRONMENT

Environment Variables

Variable	Required	Description
DATABASE_URL	Yes	PostgreSQL connection string
PORT	Yes	Port for API server (assigned by Replit)
DEFAULT_OBJECT_STORAGE_BUCKET_ID	Yes	Replit object storage bucket
PRIVATE_OBJECT_DIR	Yes	Private storage path prefix
PUBLIC_OBJECT_SEARCH_PATHS	Yes	Public storage search paths
NODE_ENV	No	development | production (defaults to development)

Startup Sequence

- 1. API server builds with esbuild (node ./build.mjs).
- 2. Server starts; ensureAdminUser() seeds the admin account if missing.
- 3. slugBackfill() ensures all businesses have URL slugs.
- 4. Express listens on PORT (default 8080).
- 5. Frontend Vite dev server starts on a separate PORT.
- 6. Replit reverse proxy routes /api/* to the API server and /* to Vite.

Production Deployment

The platform is deployed on Replit. Publishing creates a production build where Vite outputs static assets that are served by the Replit static file host. The API server runs as a persistent Node.js process. Both share the same PostgreSQL database. The production domain is mystreetly.app.

Admin Seeding

On first startup, ensureAdminUser() creates the admin account if it does not exist. Credentials are defined in the server source code. Change these immediately after first deploy in a production environment via the admin account settings.

Security Note
All sensitive configuration (database credentials, storage keys) must be stored as Replit Secrets (environment variables), never committed to source code.

pnpm Workspace Commands

Command	Description
pnpm --filter @workspace/api-server run dev	Start API server in development mode
pnpm --filter @workspace/streetly run dev	Start frontend Vite dev server
pnpm --filter @workspace/db run generate	Generate Drizzle migrations from schema
pnpm --filter @workspace/db run push	Push schema changes to database (dev only)
pnpm --filter @workspace/api-zod run codegen	Regenerate Zod schemas from OpenAPI spec

STREETLY

Street-Level Business Discovery for Nigeria

mystreetly.app